Computer Architecture
Re-take Exam

Warning: Do not fill the personal information or look at the contents of the exam before instructed by the invigilators. Violations will be treated as academic misconduct.

Personal Information

Student ID: _____

Name: _____

Family Name: _____

Signature: _____

Exam Information:

Lecturer:	Yuval Yarom
Exam Time:	14.03.2025 09:30
Semester:	Winter Semester 2024/25
Exam Length:	120 minutes

Instructions:

- Fill your personal information before starting the exam.
- Write all answers within the designated boxes on the exam paper
- If you need more space, use the extra blank pages at the end of the exam
- Examination material must not be removed from the examination room
- All answers must be in English

Materials

- One A4 reference sheet – must be handed in with exam

Marking:

Question	1	2	3	4	5	6	7
Mark							
Question	8	9	10	11	12	13	14
Mark							
Question	15	16	17	18	19	20	
Mark							

Exam: _____ /100 + Bonus: _____ = **Total:** [ ]

Question 1 — 4 marks

A *dual-issue* processor reads two consecutive instructions in each cycle, and tries to execute them in parallel. Consider a non-pipelined dual-issue processor, where all instructions execute within one cycle. In such a processor, the first instruction in each pair can always be executed immediately. However, there are cases where the processor cannot execute the second instruction or cannot execute it immediately. A rather extreme example is when the first instruction fails or causes a trap. Identify all other cases where the processor cannot execute the two instructions in parallel.

Question 2 — 4 marks

Analyzing the dual-issue processor from Question 1, we find that in a typical program, 20% of the instructions execute alone, i.e., without parallel execution. What speed-up does the dual-issue processor provide?

Please go on to the next page...

Question 3 — 4 marks

What are the smallest and largest (decimal) numbers that can be represented in 6 bits using the representations below:

	Smallest	Largest
Two's complement:		
Negabinary:		

Scratch space

Question 4 4 marks

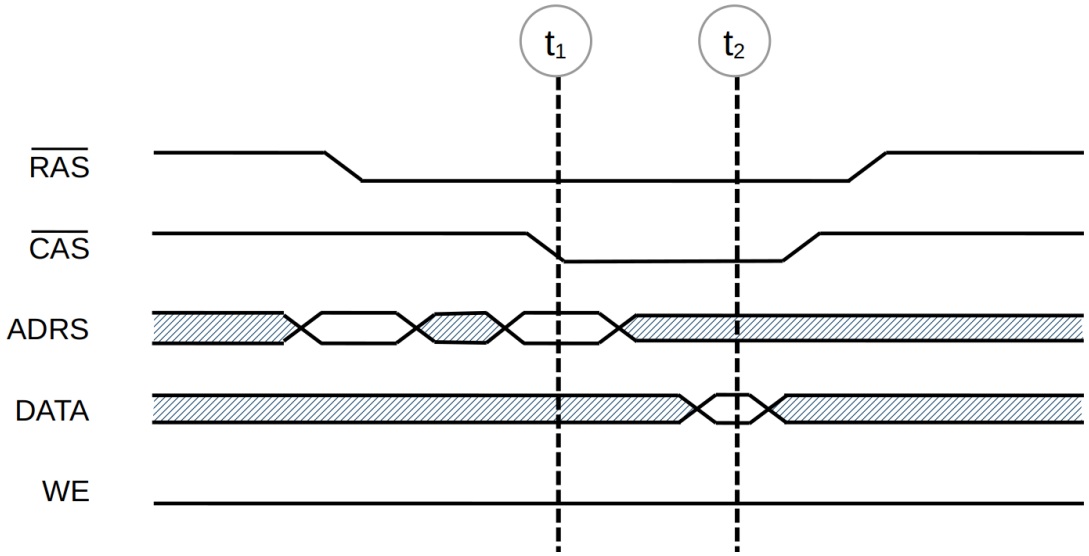
In the lectures and tutorials, we have seen some RISC-V instructions, such as Load Immediate (`li`) and Branch if Equal to Zero (`beqz`). Why do these instructions not appear in the RISC-V cheat sheet?

What are the instructions from the cheat sheet that are equivalent to:

```
li t0, 0x1000
```

```
beqz t0, end
```

Question 5 4 marks



The figure above shows a timing diagram of a DRAM transaction, with two time points, t_1 and t_2 marked. Describe what happens at times t_1 and t_2. What information is sent at each time point? Who sends that information?

Question 6 4 marks

After finishing your studies, you get a job at a start-up company in the Ruhr area. One of the main products of the company is a CPU, which features a small cache. Evaluating the cache reveals that data in the cache is served within 10 ns. However, 10% of the memory accesses result in cache misses, in which case the memory access takes 80 ns.

Your first task is to improve the performance of the cache. You redesign a new cache system that reduces the miss rate by half to 5%. However, your new cache is slightly slower, and cache hits now take 14 ns. This is achieved with no change to the memory access latency.

Does your design provide better performance? Explain your answer.

Question 7 — 4 marks

What is the purpose of **register renaming**? How does it achieve its aims?

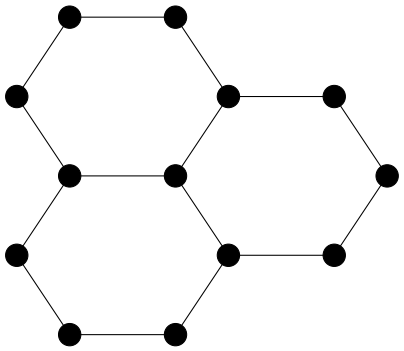
Question 8 — 4 marks

An instruction causes a divide-by-zero error during execution in a Tomasulo-based processor. At which stage is the exception detected, and how is it handled?

Question 9 — 4 marks

Assume a processor that fetches instructions at a rate of 1 million instructions per second. A detached DMA controller uses cycle stealing to copy data from a device to memory. Assuming that the device sends data at a rate of 10,000 words per second, how much would execution be slowed down when the device sends the data?

Question 10 — 4 marks



The figure above depicts a network graph G. What are

$\text{Diameter}(G)$

$\text{Degree}(G)$

$|\text{Connections}|(G)$

Question 11 — 6 marks

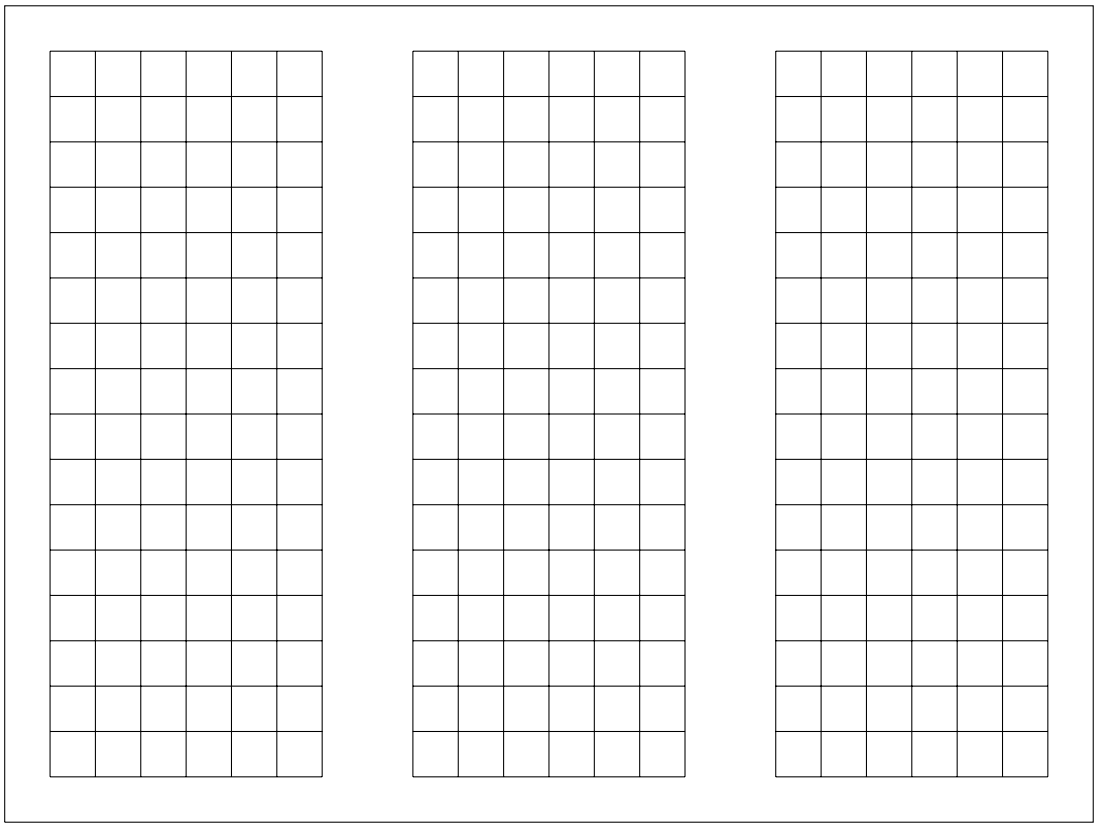
In this question we use Hamming code to encode and decode bit strings. In all tasks, bit numbers grow from right to left, with the least significant bit being number 1. For convenience, we insert spaces between every group of four bits. These spaces are **not** part of the bit stream.

(a) Encode 101 1011 0111

(b) Identify the error bit:
011 0101 1001 0100

(c) Decode the bit string from Task (b) above:

The tables below are provided for your convenience. Only the answers in the boxes above will be considered.



Question 12 — 6 marks

Fill the table to show the dependencies between the instructions in the following code

```
I1  addi  t0, t2, 7
I2  mul   t1, t2, t1
I3  sub   t2, t0, t3
I4  addi  t5, t0, 2
I5  div   t4, t6, t3
I6  slt   t3, t1, t2
I7  add   t0, t3, t5
I8  sub   t5, t0, t4
```

Use the format T(), A(), O() to represent true, anti, and output dependencies, respectively, with register name in parenthesis. In case of multiple dependencies between instructions, write all in the same table cell. The table already shows one example of a dependency.

(In case this gets messy, there is an additional copy of the table in Page 21. Please indicate clearly which copy contains your answer.)

	I1	**I2**	**I3**	**I4**	**I5**	**I6**	**I7**	**I8**
I1			T(t0)					
I2								
I3								
I4								
I5								
I6								
I7								
I8								

Question 13 — 6 marks

Determine for each of the re-ordering of the code sequence in Question 12 (reproduced here on the right) whether it is valid (✓) or not (✗) assuming register renaming. When an order is not valid, specify at least one dependency that fails.

```
I1  addi  t0, t2, 7
I2  mul   t1, t2, t1
I3  sub   t2, t0, t3
I4  addi  t5, t0, 2
I5  div   t4, t6, t3
I6  slt   t3, t1, t2
I7  add   t0, t3, t5
I8  sub   t5, t0, t4
```

i. I4, I1, I6, I2, I5, I7, I8, I3

ii. I5, I1, I4, I2, I3, I6, I7, I8

iii. I1, I2, I3, I5, I6, I4, I7, I8

iv. I1, I4, I2, I3, I5, I7, I6, I8

Scratch space

Question 14 — 6 marks

Execute the code from Question 12 in a 5-stage in-order pipeline without forwarding and without register renaming . How many NOPs are needed?

```
I1  addi  t0, t2, 7
I2  mul   t1, t2, t1
I3  sub   t2, t0, t3
I4  addi  t5, t0, 2
I5  div   t4, t6, t3
I6  slt   t3, t1, t2
I7  add   t0, t3, t5
I8  sub   t5, t0, t4
```

Question 15 6 marks

A computer has a small set-associative cache with 8 sets and 2 ways. The size of each cache line is 32 bytes. The cache uses an LRU replacement policy. Starting from an empty cache, a program accesses a sequence of addresses. For each access determine if it is a hit or a miss. For cache misses, also determine the miss type.

Write the answers in the provided boxes using the following code:

- **H** for a cache hit
- **C** for a compulsory cache miss
- **P** for a capacity cache miss
- **X** for a conflict cache miss

We marked the first three accesses.

- `0x7557` C
- `0x7557` H
- `0x32C0` C
- `0x8431`
- `0x8440`
- `0x44D0`
- `0x0352`
- `0x4232`
- `0x843f`
- `0x8450`
- `0x0358`
- `0x833f`
- `0x8420`
- `0xB5D8`

Scratch space

Question 16 6 marks

The code below takes input n and m in registers `t0` and `t1`, respectively. It then computes a function of n and m, leaving the outputs in registers `t0` and `t1`. What function does the code compute? Assuming we execute the code with inputs $n = 73$ and $m = 10$, what would the values of the output registers `t0` and `t1` be at the end of the execution?

```
    li    t0, n
    li    t1, m
    li    t2, 0
loop:
    bltu  t0, t1, end
    sub   t0, t0, t1
    addi  t2, t2, 1
    j     loop
end:
    mv    t1, t2
```

Please go on to the next page...

Question 17 6 marks

Write a RISC-V program that takes an unsigned integer n in register `t0` as input. The program outputs a number in register `t0`. The output is n if n is even. Otherwise the output is 0.

For full marks, your program should consist only of the load immediate instruction (`li`), bit shifts (`sll`, `srl`, `sra`, and their immediate variants), and logical operations (`xor`, `and`, `or`, and their immediate variants).

If your program contains an arithmetic operation (`add`, `addi`, `sub`, and any variant of `mul`, `div`, or `rem`), the maximum mark you can get for this question is five points.

If your program contains any other instruction, the maximum mark you can get for this question is four points.

Question 18 6 marks

Write a RISC-V program that takes an unsigned integer n in register `t0` as input. The program outputs a number in register `t0`. The output is $3n + 1$ if computing that value causes no overflow, and 0 if $3n + 1$ is too large, i.e. if computing it causes an overflow.

Question 19 6 marks

Write a RISC-V program that takes an unsigned integer n in register `t0` as input. The program outputs a number in register `t0`. The output is $n/2$ if n is even, and $3n + 1$ if n is odd.

You are allowed to use any RISC-V instruction and there is no need to handle overflows.

Question 20 6 marks

The Syracuse sequence of a positive integer n is defined as follows. The first element of the sequence is $S_0 = n$. Then, each element is computed from its predecessor. If S_i is even, $S_{i+1} = S_i/2$. Otherwise, if S_i is odd, $S_{i+1} = 3S_i + 1$. The sequence stops when reaching 1. For example, the Syracuse sequence of 7 is: 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1.

Write a RISC-V program that gets an unsigned integer n in register `t0` as input. The program computes the length of the Syracuse sequence of n, returning the result in `t0`. For example, if the input is 7, the output is 17, because the sequence above contains 17 numbers.

You can assume that no overflow occurs during the computation of the sequence.

Free & Open RISC-V Reference Card ①

Base Integer Instructions: RV32I, RV64I, and RV128I

Category	Name	Fmt	RV32I Base	+RV{64,128}
Loads	Load Byte	I	`LB rd,rs1,imm`	
	Load Halfword	I	`LH rd,rs1,imm`	
	Load Word	I	`LW rd,rs1,imm`	`L{D|Q} rd,rs1,imm`
	Load Byte Unsigned	I	`LBU rd,rs1,imm`	
	Load Half Unsigned	I	`LHU rd,rs1,imm`	`L{W|D}U rd,rs1,imm`
Stores	Store Byte	S	`SB rs1,rs2,imm`	
	Store Halfword	S	`SH rs1,rs2,imm`	
	Store Word	S	`SW rs1,rs2,imm`	`S{D|Q} rs1,rs2,imm`
Shifts	Shift Left	R	`SLL rd,rs1,rs2`	`SLL{W|D} rd,rs1,rs2`
	Shift Left Immediate	I	`SLLI rd,rs1,shamt`	`SLLI{W|D} rd,rs1,shamt`
	Shift Right	R	`SRL rd,rs1,rs2`	`SRL{W|D} rd,rs1,rs2`
	Shift Right Immediate	I	`SRLI rd,rs1,shamt`	`SRLI{W|D} rd,rs1,shamt`
	Shift Right Arithmetic	R	`SRA rd,rs1,rs2`	`SRA{W|D} rd,rs1,rs2`
	Shift Right Arith Imm	I	`SRAI rd,rs1,shamt`	`SRAI{W|D} rd,rs1,shamt`
Arithmetic	ADD	R	`ADD rd,rs1,rs2`	`ADD{W|D} rd,rs1,rs2`
	ADD Immediate	I	`ADDI rd,rs1,imm`	`ADDI{W|D} rd,rs1,imm`
	SUBtract	R	`SUB rd,rs1,rs2`	`SUB{W|D} rd,rs1,rs2`
	Load Upper Imm	U	`LUI rd,imm`	
	Add Upper Imm to PC	U	`AUIPC rd,imm`	
Logical	XOR	R	`XOR rd,rs1,rs2`	
	XOR Immediate	I	`XORI rd,rs1,imm`	
	OR	R	`OR rd,rs1,rs2`	
	OR Immediate	I	`ORI rd,rs1,imm`	
	AND	R	`AND rd,rs1,rs2`	
	AND Immediate	I	`ANDI rd,rs1,imm`	
Compare	Set <	R	`SLT rd,rs1,rs2`	
	Set < Immediate	I	`SLTI rd,rs1,imm`	
	Set < Unsigned	R	`SLTU rd,rs1,rs2`	
	Set < Imm Unsigned	I	`SLTIU rd,rs1,imm`	
Branches	Branch =	SB	`BEQ rs1,rs2,imm`	
	Branch ≠	SB	`BNE rs1,rs2,imm`	
	Branch <	SB	`BLT rs1,rs2,imm`	
	Branch ≥	SB	`BGE rs1,rs2,imm`	
	Branch < Unsigned	SB	`BLTU rs1,rs2,imm`	
	Branch ≥ Unsigned	SB	`BGEU rs1,rs2,imm`	
Jump & Link	J&L	UJ	`JAL rd,imm`	
	Jump & Link Register	UJ	`JALR rd,rs1,imm`	
Synch	Synch thread	I	`FENCE`	
	Synch Instr & Data	I	`FENCE.I`	
System	System CALL	I	`SCALL`	
	System BREAK	I	`SBREAK`	
Counters	ReaD CYCLE	I	`RDCYCLE rd`	
	ReaD CYCLE upper Half	I	`RDCYCLEH rd`	
	ReaD TIME	I	`RDTIME rd`	
	ReaD TIME upper Half	I	`RDTIMEH rd`	
	ReaD INSTR RETired	I	`RDINSTRET rd`	
	ReaD INSTR upper Half	I	`RDINSTRETH rd`	

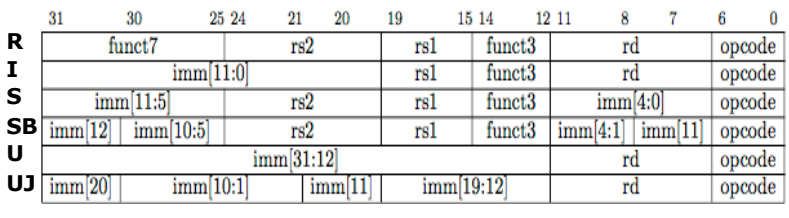
RV Privileged Instructions

Category	Name	RV mnemonic
CSR Access	Atomic R/W	`CSRRW rd,csr,rs1`
	Atomic Read & Set Bit	`CSRRS rd,csr,rs1`
	Atomic Read & Clear Bit	`CSRRC rd,csr,rs1`
	Atomic R/W Imm	`CSRRWI rd,csr,imm`
	Atomic Read & Set Bit Imm	`CSRRSI rd,csr,imm`
	Atomic Read & Clear Bit Imm	`CSRRCI rd,csr,imm`
Change Level	Env. Call	`ECALL`
	Environment Breakpoint	`EBREAK`
	Environment Return	`ERET`
Trap Redirect	to Supervisor	`MRTS`
	Redirect Trap to Hypervisor	`MRTH`
	Hypervisor Trap to Supervisor	`HRTS`
Interrupt	Wait for Interrupt	`WFI`
MMU	Supervisor FENCE	`SFENCE.VM rs1`

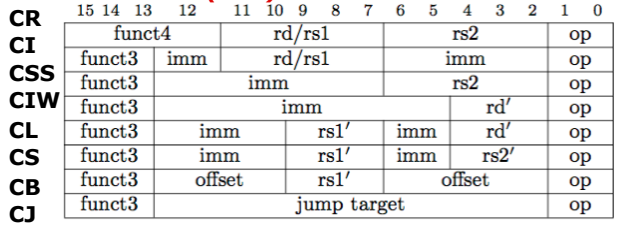
Optional Compressed (16-bit) Instruction Extension: RVC

Category	Name	Fmt	RVC	RVI equivalent
Loads	Load Word	CL	`C.LW rd',rs1',imm`	`LW rd',rs1',imm*4`
	Load Word SP	CI	`C.LWSP rd,imm`	`LW rd,sp,imm*4`
	Load Double	CL	`C.LD rd',rs1',imm`	`LD rd',rs1',imm*8`
	Load Double SP	CI	`C.LDSP rd,imm`	`LD rd,sp,imm*8`
	Load Quad	CL	`C.LQ rd',rs1',imm`	`LQ rd',rs1',imm*16`
	Load Quad SP	CI	`C.LQSP rd,imm`	`LQ rd,sp,imm*16`
Stores	Store Word	CS	`C.SW rs1',rs2',imm`	`SW rs1',rs2',imm*4`
	Store Word SP	CSS	`C.SWSP rs2,imm`	`SW rs2,sp,imm*4`
	Store Double	CS	`C.SD rs1',rs2',imm`	`SD rs1',rs2',imm*8`
	Store Double SP	CSS	`C.SDSP rs2,imm`	`SD rs2,sp,imm*8`
	Store Quad	CS	`C.SQ rs1',rs2',imm`	`SQ rs1',rs2',imm*16`
	Store Quad SP	CSS	`C.SQSP rs2,imm`	`SQ rs2,sp,imm*16`
Arithmetic	ADD	CR	`C.ADD rd,rs1`	`ADD rd,rd,rs1`
	ADD Word	CR	`C.ADDW rd,rs1`	`ADDW rd,rd,imm`
	ADD Immediate	CI	`C.ADDI rd,imm`	`ADDI rd,rd,imm`
	ADD Word Imm	CI	`C.ADDIW rd,imm`	`ADDIW rd,rd,imm`
	ADD SP Imm * 16	CI	`C.ADDI16SP x0,imm`	`ADDI sp,sp,imm*16`
	ADD SP Imm * 4	CIW	`C.ADDI4SPN rd',imm`	`ADDI rd',sp,imm*4`
	Load Immediate	CI	`C.LI rd,imm`	`ADDI rd,x0,imm`
	Load Upper Imm	CI	`C.LUI rd,imm`	`LUI rd,imm`
	MoVe	CR	`C.MV rd,rs1`	`ADD rd,rs1,x0`
	SUB	CR	`C.SUB rd,rs1`	`SUB rd,rd,rs1`
Shifts	Shift Left Imm	CI	`C.SLLI rd,imm`	`SLLI rd,rd,imm`
Branches	Branch=0	CB	`C.BEQZ rs1',imm`	`BEQ rs1',x0,imm`
	Branch≠0	CB	`C.BNEZ rs1',imm`	`BNE rs1',x0,imm`
Jump	Jump	CJ	`C.J imm`	`JAL x0,imm`
	Jump Register	CR	`C.JR rd,rs1`	`JALR x0,rs1,0`
Jump & Link	J&L	CJ	`C.JAL imm`	`JAL ra,imm`
	Jump & Link Register	CR	`C.JALR rs1`	`JALR ra,rs1,0`
System	Env. BREAK	CI	`C.EBREAK`	`EBREAK`

32-bit Instruction Formats



16-bit (RVC) Instruction Formats



RISC-V Integer Base (RV32I/64I/128I), privileged, and optional compressed extension (RVC). Registers x1-x31 and the pc are 32 bits wide in RV32I, 64 in RV64I, and 128 in RV128I (x0=0). RV64I/128I add 10 instructions for the wider formats. The RVI base of <50 classic integer RISC instructions is required. Every 16-bit RVC instruction matches an existing 32-bit RVI instruction. See risc.org.

Free & Open RISC-V Reference Card (riscv.org) ②

Optional Multiply-Divide Instruction Extension: RVM

Category	Name	Fmt	RV32M (Multiply-Divide)		+RV{64,128}	
Multiply	MULtiply	R	MUL	rd,rs1,rs2	MUL{W|D}	rd,rs1,rs2
	MULtiply upper Half	R	MULH	rd,rs1,rs2		
	MULtiply Half Sign/Uns	R	MULHSU	rd,rs1,rs2		
	MULtiply upper Half Uns	R	MULHU	rd,rs1,rs2		
Divide	DIVide	R	DIV	rd,rs1,rs2	DIV{W|D}	rd,rs1,rs2
	DIVide Unsigned	R	DIVU	rd,rs1,rs2		
Remainder	REMainder	R	REM	rd,rs1,rs2	REM{W|D}	rd,rs1,rs2
	REMainder Unsigned	R	REMU	rd,rs1,rs2	REMU{W|D}	rd,rs1,rs2

Optional Atomic Instruction Extension: RVA

Category	Name	Fmt	RV32A (Atomic)		+RV{64,128}	
Load	Load Reserved	R	LR.W	rd,rs1	LR.{D|Q}	rd,rs1
Store	Store Conditional	R	SC.W	rd,rs1,rs2	SC.{D|Q}	rd,rs1,rs2
Swap	SWAP	R	AMOSWAP.W	rd,rs1,rs2	AMOSWAP.{D|Q}	rd,rs1,rs2
Add	ADD	R	AMOADD.W	rd,rs1,rs2	AMOADD.{D|Q}	rd,rs1,rs2
Logical	XOR	R	AMOXOR.W	rd,rs1,rs2	AMOXOR.{D|Q}	rd,rs1,rs2
	AND	R	AMOAND.W	rd,rs1,rs2	AMOAND.{D|Q}	rd,rs1,rs2
	OR	R	AMOOR.W	rd,rs1,rs2	AMOOR.{D|Q}	rd,rs1,rs2
Min/Max	MINimum	R	AMOMIN.W	rd,rs1,rs2	AMOMIN.{D|Q}	rd,rs1,rs2
	MAXimum	R	AMOMAX.W	rd,rs1,rs2	AMOMAX.{D|Q}	rd,rs1,rs2
	MINimum Unsigned	R	AMOMINU.W	rd,rs1,rs2	AMOMINU.{D|Q}	rd,rs1,rs2
	MAXimum Unsigned	R	AMOMAXU.W	rd,rs1,rs2	AMOMAXU.{D|Q}	rd,rs1,rs2

Three Optional Floating-Point Instruction Extensions: RVF, RVD, & RVQ

Category	Name	Fmt	RV32{F|D|Q} (HP/SP,DP,QP Fl Pt)		+RV{64,128}	
Move	Move from Integer	R	FMV.{H|S}.X	rd,rs1	FMV.{D|Q}.X	rd,rs1
	Move to Integer	R	FMV.X.{H|S}	rd,rs1	FMV.X.{D|Q}	rd,rs1
Convert	Convert from Int	R	FCVT.{H|S|D|Q}.W	rd,rs1	FCVT.{H|S|D|Q}.{L|T}	rd,rs1
	Convert from Int Unsigned	R	FCVT.{H|S|D|Q}.WU	rd,rs1	FCVT.{H|S|D|Q}.{L|T}U	rd,rs1
	Convert to Int	R	FCVT.W.{H|S|D|Q}	rd,rs1	FCVT.{L|T}.{H|S|D|Q}	rd,rs1
	Convert to Int Unsigned	R	FCVT.WU.{H|S|D|Q}	rd,rs1	FCVT.{L|T}U.{H|S|D|Q}	rd,rs1
Load	Load	I	FL{W,D,Q}	rd,rs1,imm		
Store	Store	S	FS{W,D,Q}	rs1,rs2,imm		
Arithmetic	ADD	R	FADD.{S|D|Q}	rd,rs1,rs2		
	SUBtract	R	FSUB.{S|D|Q}	rd,rs1,rs2		
	MULtiply	R	FMUL.{S|D|Q}	rd,rs1,rs2		
	DIVide	R	FDIV.{S|D|Q}	rd,rs1,rs2		
	SQuare RooT	R	FSQRT.{S|D|Q}	rd,rs1		
Mul-Add	Multiply-ADD	R	FMADD.{S|D|Q}	rd,rs1,rs2,rs3		
	Multiply-SUBtract	R	FMSUB.{S|D|Q}	rd,rs1,rs2,rs3		
	Negative Multiply-SUBtract	R	FNMSUB.{S|D|Q}	rd,rs1,rs2,rs3		
	Negative Multiply-ADD	R	FNMADD.{S|D|Q}	rd,rs1,rs2,rs3		
Sign Inject	SiGN source	R	FSGNJ.{S|D|Q}	rd,rs1,rs2		
	Negative SiGN source	R	FSGNJN.{S|D|Q}	rd,rs1,rs2		
	Xor SiGN source	R	FSGNJX.{S|D|Q}	rd,rs1,rs2		
Min/Max	MINimum	R	FMIN.{S|D|Q}	rd,rs1,rs2		
	MAXimum	R	FMAX.{S|D|Q}	rd,rs1,rs2		
Compare	Compare Float =	R	FEQ.{S|D|Q}	rd,rs1,rs2		
	Compare Float <	R	FLT.{S|D|Q}	rd,rs1,rs2		
	Compare Float ≤	R	FLE.{S|D|Q}	rd,rs1,rs2		
Categorization	Classify Type	R	FCLASS.{S|D|Q}	rd,rs1		
Configuration	Read Status	R	FRCSR	rd		
	Read Rounding Mode	R	FRRM	rd		
	Read Flags	R	FRFLAGS	rd		
	Swap Status Reg	R	FSCSR	rd,rs1		
	Swap Rounding Mode	R	FSRM	rd,rs1		
	Swap Flags	R	FSFLAGS	rd,rs1		
	Swap Rounding Mode Imm	I	FSRMI	rd,imm		
	Swap Flags Imm	I	FSFLAGSI	rd,imm		

RISC-V Calling Convention

Register	ABI Name	Saver	Description
x0	zero	---	Hard-wired zero
x1	ra	Caller	Return address
x2	sp	Callee	Stack pointer
x3	gp	---	Global pointer
x4	tp	---	Thread pointer
x5-7	t0-2	Caller	Temporaries
x8	s0/fp	Callee	Saved register/frame pointer
x9	s1	Callee	Saved register
x10-11	a0-1	Caller	Function arguments/return values
x12-17	a2-7	Caller	Function arguments
x18-27	s2-11	Callee	Saved registers
x28-31	t3-t6	Caller	Temporaries
f0-7	ft0-7	Caller	FP temporaries
f8-9	fs0-1	Callee	FP saved registers
f10-11	fa0-1	Caller	FP arguments/return values
f12-17	fa2-7	Caller	FP arguments
f18-27	fs2-11	Callee	FP saved registers
f28-31	ft8-11	Caller	FP temporaries

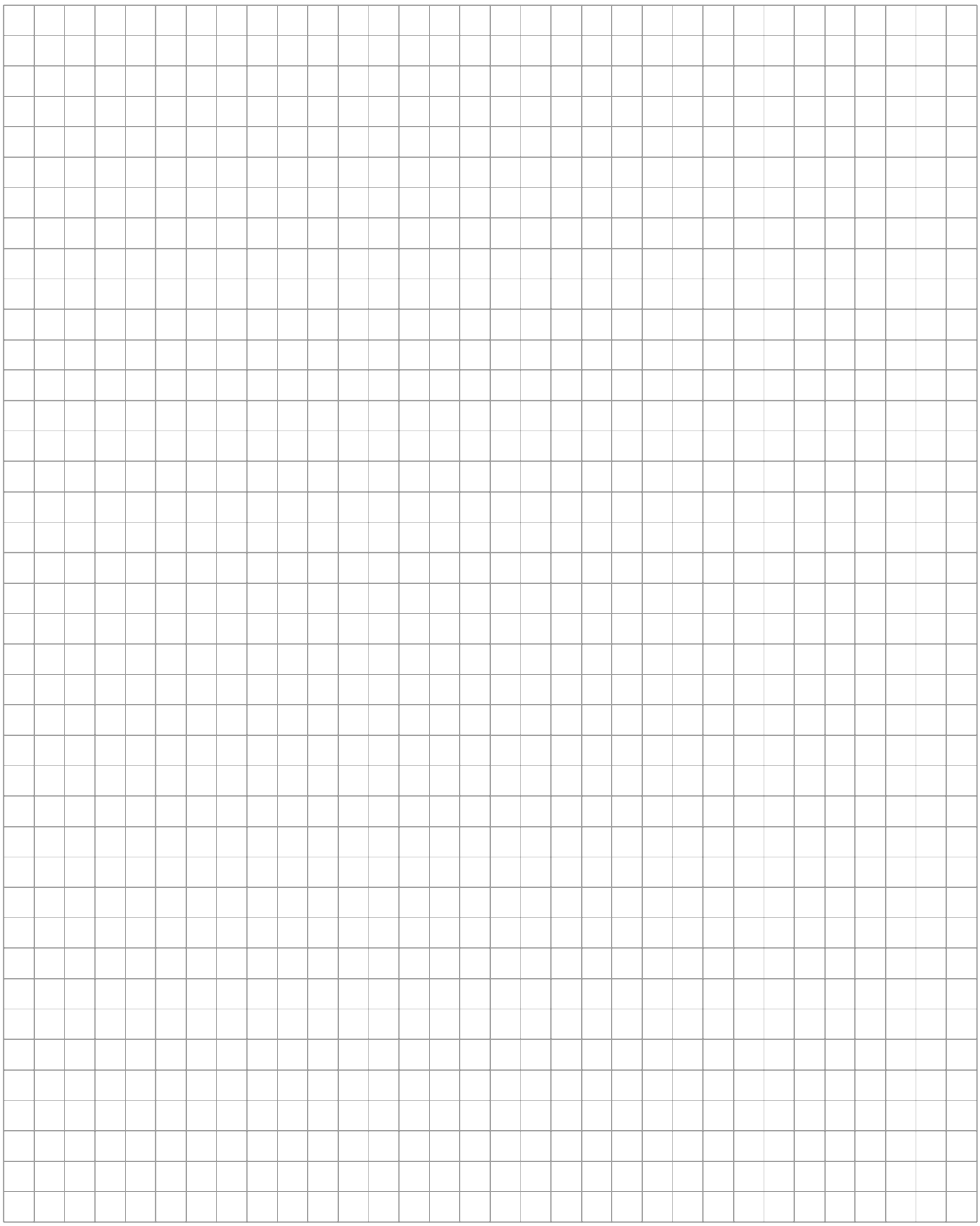
RISC-V calling convention and five optional extensions: 10 multiply-divide instructions (RV32M); 11 optional atomic instructions (RV32A); and 25 floating-point instructions each for single-, double-, and quadruple-precision (RV32F, RV32D, RV32Q). The latter add registers f0-f31, whose width matches the widest precision, and a floating-point control and status register fcsr. Each larger address adds some instructions: 4 for RVM, 11 for RVA, and 6 each for RVF/D/Q. Using regex notation, { } means set, so `L{D|Q}` *is both* `LD` *and* `LQ`*. See risc.org. (8/21/15 revision)*

Extra table for Question 12

Please indicate clearly which copy contains your answer.

	I1	**I2**	**I3**	**I4**	**I5**	**I6**	**I7**	**I8**
I1			T(t0)					
I2								
I3								
I4								
I5								
I6								
I7								
I8								

Extra Space for Answers
Please indicate the question number clearly



Extra Space for Answers
Please indicate the question number clearly

Extra Space for Answers
Please indicate the question number clearly

End of exam